ВВЕДЕНИЕВ современных условиях деятельность образовательных центров сопровождается ростом объемов обрабатываемой информации, включающей данные об обучающихся, расписании занятий и результатах успеваемости. Традиционные методы ведения документации, предполагающие использование бумажных журналов, несистематизированных электронных таблиц и других форм учета, характеризуются низкой эффективностью, высокой вероятностью возникновения ошибок и отсутствием возможности оперативного взаимодействия между администрацией, преподавателями и учениками.

Проблема заключается в противоречии между возрастающими требованиями к качеству сервиса в образовательной среде и недостатком гибких инструментов для автоматизации. Актуальность работы обусловлена потребностью в создании адаптивного программного обеспечения, отвечающего требованиям образовательного центра. В отличие от типовых решений, разработка собственной информационной системы позволяет реализовать функционал, максимально соответствующий текущим бизнес-процессам и особенностям организационной структуры.

Целью данной работы является разработка и внедрение информационной системы, направленной на автоматизацию ключевых бизнес-процессов образовательного центра.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и обосновать функциональные требования к информационной системе образовательного центра.
2. Спроектировать архитектуру информационной системы и разработать реляционную модель базы данных, обеспечивающую целостность хранимой информации.
3. Осуществить программную реализацию функциональных модулей системы, включая механизмы авторизации пользователей и разграничения прав доступа.
4. Провести тестирование разработанной информационной системы.

Объектом исследования являются бизнес-процессы образовательного центра, связанные с организацией учебной деятельности.

Предметом исследования выступает процесс автоматизации управления учебными данными с применением современных веб-технологий.

ГЛАВА I. ТЕОРЕТИЧЕСКИЕ И МЕТОДОЛОГИЧЕСКИЕ ОСНОВЫ АВТОМАТИЗАЦИИ УПРАВЛЕНИЯ ОБРАЗОВАТЕЛЬНЫМ ПРОЦЕССОМ1.1. Роль и место информационных систем в управлении образовательным центромСовременная образовательная организация представляет собой сложную социально-техническую систему, эффективность функционирования которой напрямую зависит от качества управления информационными потоками. В условиях цифровизации экономики образовательный сектор сталкивается с необходимостью трансформации внутренних бизнес-процессов.

1.1.1. Характеристика объекта автоматизацииОбразовательный центр, выступающий объектом данного исследования, реализует широкий спектр услуг, требующих постоянного учета: от формирования групп до контроля успеваемости. В текущем состоянии процессы управления характеризуются высокой долей ручного труда.

К основным бизнес-процессам, подлежащим автоматизации, относятся:

- Управление кадровым и учебным составом: ведение реестра сотрудников и обучающихся, распределение ролей в системе.
- Планирование учебной нагрузки: формирование расписания занятий, исключающее временные накладки и учитывающее занятость аудиторного фонда.
- Мониторинг процесса обучения: фиксация посещаемости, оперативное отражение изменений в учебном плане.

1.1.2. Анализ нормативно-правовых аспектовОсобое внимание при проектировании информационной системы уделяется соблюдению законодательства Российской Федерации. Согласно Федеральному закону № 152-ФЗ «О персональных данных», любая система, обрабатывающая сведения о физических лицах (сотрудниках, учениках), должна обеспечивать надлежащий уровень защиты информации.

Внедрение разрабатываемой системы позволяет реализовать требования закона за счет:

1. Идентификации и аутентификации: доступ к данным предоставляется только авторизованным пользователям.
2. Разграничения прав доступа: реализация модели RBAC (Role-Based Access Control) гарантирует, что пользователь имеет доступ только к тем данным, которые необходимы для выполнения его служебных обязанностей.
3. Управление через логирование: Запись ключевых системных операций позволяет проводить детальный аудит действий пользователей, что является важным фактором для повышения ответственности персонала, а значит, и для эффективности работы.

1.1.3. Исследование неэффективных зон в системе управленияПроведенный анализ деятельности центра позволил классифицировать основные проблемы, препятствующие эффективному развитию:

- Проблемы с доступом к информации: Отсутствие единого источника информации приводит к тому, что сотрудники работают с противоречивыми данными. Это напрямую сказывается на качестве планирования нагрузки, вызывая ошибки.
- Значительный объем рабочего времени администратора уходит на обработку стандартных запросов от преподавателей, касающихся расписания занятий и информации об учениках и учебных групп.
- Отсутствие своевременной аналитической информации, такой как отчеты о посещаемости за месяц, ограничивает способность руководства к принятию обоснованных решений.

Таким образом, переход к автоматизированной системе управления является не просто техническим обновлением, а стратегическим шагом, направленным на повышение конкурентоспособности образовательного центра на рынке услуг. Проектируемая система призвана стать инструментом, который минимизирует влияние «человеческого фактора» на качество предоставляемых данных и оптимизирует взаимодействие всех участников образовательного процесса.

1.2. Моделирование и анализ бизнес-процессов образовательного центраДля эффективной автоматизации необходимо формализовать последовательность действий, составляющих учебную деятельность. В рамках исследования были выделены три ключевых бизнес-процесса: «Управление кадрами», «Управление учебной нагрузкой» и «Мониторинг успеваемости».

1.2.1. Описание основных бизнес-процессов1. Процесс «Администрирование пользователей»: включает создание учетных записей, назначение ролей (преподаватель, ученик) и управление филиальной структурой. В текущих условиях данный процесс осложнен необходимостью ведения ручных реестров.
2. Процесс «Планирование учебного расписания»: является наиболее трудоемким. Он включает сбор данных о доступности преподавателей, наличие свободных аудиторий и распределение их по группам. Автоматизация данного процесса исключает возможность «накладок» в расписании, свойственных при использовании табличных методов.
3. Процесс «Мониторинг посещаемости»: процесс фиксации присутствия обучающихся на занятиях. Автоматизация позволяет перевести данный процесс в электронный формат, обеспечивая преподавателю быстрый доступ к журналу, а администрации — актуальную аналитику.

1.2.2. Анализ «узких мест» (AS-IS)В текущем состоянии («AS-IS» — как есть) процессы имеют следующие недостатки:

- Информационная избыточность: данные передаются между участниками через неформализованные каналы связи (мессенджеры, электронная почта), что ведет к потере актуальности данных.
- Отсутствие контроля доступа: информация об успеваемости и личные данные обучающихся зачастую доступны сотрудникам, не участвующим в учебном процессе напрямую, что нарушает принципы информационной безопасности.
- Высокая вероятность ошибок при вводе: отсутствие системы валидации данных при ручном заполнении журналов приводит к возникновению артефактов и неточностей.

1.2.3. Целевая модель бизнес-процессов (TO-BE)Переход к целевой модели («TO-BE» — как должно быть) после внедрения ИС предполагает:

- Централизацию: создание единой базы данных SQLite, доступ к которой осуществляется через авторизованный интерфейс.
- Автоматизацию контроля: система автоматически проверяет корректность введенных данных (например, невозможность назначения двух занятий на одно время для одной группы).
- Разграничение прав: реализация ролевой модели доступа обеспечивает безопасность данных в соответствии с ФЗ-152.

1.3. Сравнительный анализ подходов к автоматизации учебной деятельностиДля выбора оптимального решения задачи автоматизации был проведен анализ существующих подходов. Сравнительная характеристика представлена в таблице

Анализ показывает, что облачные CRM-системы часто перегружены избыточным функционалом, что усложняет процесс их внедрения в деятельность малых образовательных центров. Разрабатываемая информационная система ориентирована на специфические нужды заказчика, что обеспечивает баланс между функциональностью и простотой сопровождения.

1.4. Методология проектирования и жизненный цикл разработки системыВ качестве методологии проектирования информационной системы был выбран итеративный подход. Данная стратегия разработки позволяет реализовывать функциональные модули поэтапно, что обеспечивает возможность оперативной корректировки архитектуры системы в соответствии с возникающими требованиями.

Жизненный цикл разработки (SDLC) включил следующие этапы:

1. Анализ требований: выявление функциональных потребностей пользователей (администратор, преподаватель, ученик).
2. Проектирование архитектуры: Разработка структуры базы данных и модели взаимодействия клиент-сервер.
3. Реализация: программная разработка серверной логики на языке Python с использованием фреймворка Flask.
4. Тестирование: оценка корректности ролевой модели доступа и надежности системы при обработке некорректных пользовательских запросов.

1.5. Обоснование выбора программно-аппаратного стека технологийВыбор инструментов разработки подобран с учетом требований к масштабируемости, отказоустойчивости и эффективности работы.

Серверная логика и фреймворк: Использование веб-фреймворка Flask обусловлено его архитектурной гибкостью. Применение модульного подхода (Blueprint) позволило разделить систему на функциональные блоки, что обеспечивает поддержку и масштабируемость системы в будущем.

Управление данными: Выбор реляционной базы данных SQLite обусловлен требованиями к автономности и отсутствию необходимости в настройке выделенного сервера БД. Интеграция с библиотекой SQLAlchemy обеспечивает эффективное объектно-реляционное отображение, позволяя выполнять операции с системными данными на уровне Python-объектов.

Интерфейс пользователя: Реализация клиентской части на основе HTML5 и CSS3 в сочетании с шаблонизатором Jinja2 позволила обеспечить адаптивность интерфейса, что является критически важным требованием для современных веб-приложений.

1.6. Стандарты качества и надежности информационных системПри проектировании информационных систем в сфере образования принципиально важным является соблюдение общепринятых критериев качества программного обеспечения (ПО). В соответствии с международным стандартом ISO/IEC 25010 и отечественными требованиями ГОСТ Р ИСО/МЭК 25010–2015, к разрабатываемой системе предъявляется ряд ключевых требований, обеспечивающих её надежность и долговечность.

1.6.1. Критерии оценки качества программного продуктаКлючевыми характеристиками, определяющими эффективность создаваемого решения, являются:

1. Функциональная полнота: способность системы обеспечивать выполнение всех бизнес-процессов, описанных в техническом задании (управление пользователями, учет посещаемости, формирование расписания).
2. Удобство использования (Usability): интерфейс должен быть интуитивно понятным для пользователей с различным уровнем технической подготовки. В данной работе это достигается за счет адаптивной верстки (HTML5/CSS3) и минимизации количества шагов для выполнения целевых действий.
3. Сопровождаемость: использование модульной архитектуры (Blueprints) и системы ORM (SQLAlchemy) позволяет вносить изменения в логику работы без необходимости полной переработки приложения.
4. Надежность: способность системы корректно обрабатывать ошибки ввода данных и восстанавливаться после программных сбоев, что достигается за счет реализации серверной валидации данных.

1.6.2. Обеспечение безопасности и целостности данныхОсобый аспект качества ИС — информационная безопасность. В контексте работы образовательного центра она включает:

- Аутентификация и авторизация: использование хэширования паролей (библиотека Werkzeug) гарантирует, что даже при компрометации базы данных пароли пользователей не будут доступны в открытом виде.
- Целостность данных (ACID): использование SQLite в связке с SQLAlchemy обеспечивает соблюдение транзакционной целостности. Например, при записи посещаемости система гарантирует, что данные будут сохранены корректно, либо операция будет отменена полностью (в случае ошибки).
- Защита от несанкционированного доступа: реализация декораторов контроля доступа (@login_required) на уровне серверных маршрутов исключает возможность обращения к закрытым данным через прямое введение URL в адресную строку браузера.

1.6.3. Экономическая эффективность разработкиВнедрение автоматизированной системы позволяет образовательному центру достичь значительной экономической эффективности:

- Оптимизация операционных расходов: Автоматизация рутинных административных процессов, таких как поиск информации, формирование бумажных отчетов, высвобождает ресурсы, которые могут быть направлены на развитие и реализацию основной образовательной деятельности.
- Оптимизация фонда рабочего времени: Внедрение автоматизированных систем оптимизирует использование рабочего времени, переориентируя персонал с документационного оборота на непосредственное взаимодействие с учащимися.
- Минимизация потерь: за счет исключения ошибок, свойственных ручному вводу данных, повышается достоверность управленческой отчетности.

1.7 Выводы по первой главеВ первой главе были рассмотрены теоретические основы автоматизации деятельности образовательного центра. Проведенный анализ предметной области позволил выявить ключевые проблемы: фрагментарность данных, отсутствие оперативного взаимодействия) и обосновать необходимость внедрения системы управления.

Методологические аспекты, такие как итеративная модель разработки и выбор стека технологий (Python/Flask), были подтверждены требованиями к масштабируемости и надежности ПО. Соответствие системы стандартам качества ISO/IEC 25010 гарантирует, что разработанный продукт не только решает текущие задачи центра, но и обладает необходимым потенциалом для дальнейшего развития.

ГЛАВА II. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ2.1. Проектирование логической модели и архитектуры базы данных2.1.1. Обоснование выбора и проектирование системы хранения данныхЭффективность функционирования информационной системы напрямую коррелирует с архитектурой её подсистемы хранения данных. Для образовательного центра была выбрана реляционная модель, так как бизнес-процессы (учет посещаемости, закрепление преподавателей за группами, формирование расписания) обладают четко выраженной структурой и строгими правилами взаимосвязи между объектами.

В качестве системы управления базами данных (СУБД) была выбрана SQLite. Выбор данного решения обусловлен тремя ключевыми факторами:

1. Автономность: Отсутствие необходимости в выделенном сервере БД снижает требования к хостингу и упрощает развертывание системы. В условиях малого бизнеса это позволяет исключить избыточные затраты на администрирование серверной инфраструктуры.
2. Целостность: СУБД поддерживает транзакции (ACID-принципы — атомарность, согласованность, изолированность, долговечность). Данный подход позволяет избежать нарушения структуры данных в случае возникновения внештатных ситуаций, таких как программные сбои или прерывание связи между клиентом и сервером. Транзакционная логика гарантирует, что операция записи статуса посещаемости выполняется как единое, неделимое действие: в случае невозможности завершения операции база данных возвращается к исходному состоянию, исключая возможность сохранения некорректных или фрагментарных записей.
3. Интеграция SQLAlchemy ORM: Взаимодействие с базой данных в системе реализовано посредством библиотеки SQLAlchemy. Использование уровня объектно-реляционного отображения (ORM) позволяет перенести операции обработки данных с использования прямых SQL-запросов на уровень объектной модели языка Python. Данное решение способствует повышению уровня сопровождаемости программного продукта: при необходимости корректировки бизнес-логики разработчику достаточно изменить структуру соответствующего класса в файле models.py, вместо внесения правок в массив SQL-инструкций. Применение ORM-подхода минимизирует риск возникновения ошибок, обусловленных некорректным синтаксисом запросов, и обеспечивает унификацию способов работы с данными в рамках всей системы.

2.1.2. Описание структурной схемы базы данныхВ ходе проектирования архитектуры базы данных приоритетной задачей стала разработка такой структуры, которая позволила бы эффективно хранить иерархические связи образовательного процесса. Кроме того, к модели предъявлялись требования по обеспечению механизмов разграничения прав доступа на уровне СУБД, что необходимо для защиты конфиденциальных сведений об участниках обучения. Итоговая модель данных позволяет организовать хранение взаимосвязанных сущностей таким образом, чтобы обеспечить логическую непротиворечивость данных при выполнении операций записи, редактирования и поиска

Описание ключевых сущностей представлено в таблице 2.1.

Таблица 2.1 Характеристика ключевых сущностей базы данных

Для поддержания корректности структуры базы данных при проектировании было выполнено приведение модели к третьей нормальной форме (3NF), что исключает избыточность данных и связанные с ней аномалии обновления. Механизмы взаимодействия между таблицами (Relationships) настроены в соответствии со следующими принципами проектирования реляционных баз данных:

- Связь «один-ко-многим» (1:M) применена для моделирования иерархических зависимостей, таких как распределение студентов по учебным группам и назначение преподавателя на дисциплины. Техническая реализация данного типа отношений выполнена посредством внешних ключей (ForeignKey). Данный механизм обеспечивает ссылочную целостность базы данных, исключая возможность возникновения логических ошибок — в частности, система программно блокирует создание записи о назначении преподавателя на несуществующий в базе объект занятия
- Для реализации связи «многие-ко-многим» (M:N) была внедрена промежуточная таблица Attendance. Данная архитектурная единица позволяет реализовать связь между множеством студентов и множеством учебных занятий. Использование такой структуры дает возможность не только обеспечить связность данных, но и хранить атрибутивную информацию — в частности, статус посещения (status) для каждой уникальной пары «студент–занятие», что является необходимым условием для формирования отчетности по успеваемости
- Каскадные операции: для поддержания ссылочной целостности базы данных при удалении родительских сущностей (таких как филиалы, учебные группы или записи о сотрудниках) настроены правила каскадного выполнения операций (CASCADE). Использование данных механизмов позволяет автоматизировать процесс очистки зависимых данных в связанных таблицах. Это исключает возникновение в системе некорректных записей, не имеющих ссылок на существующие объекты, и избавляет от необходимости выполнения ручной корректировки базы данных при изменении организационной структуры образовательного центра.

2.2. Программная реализация функциональных модулейАрхитектура системы разработана на принципах модульности и возможности дальнейшего масштабирования. Использование механизма Blueprints во фреймворке Flask позволило разделить приложение на независимые компоненты, каждый из которых инкапсулирует бизнес-логику определенного функционального блока.

2.2.1. Модульная архитектура приложения на основе BlueprintsВ целях повышения сопровождаемости и обеспечения масштабируемости программного обеспечения архитектура системы была спроектирована по принципу декомпозиции на слабосвязанные функциональные модули. В качестве инструментального решения для реализации данного подхода выбран механизм Blueprints (чертежей), предусмотренный фреймворком Flask для организации структуры крупных приложений. Использование данного паттерна позволяет выделить автономные компоненты системы, каждый из которых инкапсулирует специфическую бизнес-логику и взаимодействует с другими частями приложения через стандартизированные интерфейсы.

Применение механизма Blueprints позволило структурировать систему путем выделения независимых логических блоков:

- Модуль аутентификации (auth_bp): предназначен для управления жизненным циклом пользовательских сессий и процедурами верификации учетных данных. Выделение данного функционала в отдельный блок обеспечивает возможность централизованной настройки политик безопасности.
- Модуль администрирования (admin_bp): обеспечивает инструментарий для управления системными ресурсами и кадровым учетом организации. Данный модуль позволяет разграничить права доступа и логику работы. Это обеспечивает возможность изменять параметры учебного процесса (структуру филиалов, состав групп или правила формирования расписания) независимо от основного функционала. Таким образом, административные задачи выполняются в изолированном контуре, что исключает сбои в работе интерфейса преподавателей при внесении изменений в настройки системы
- Модуль преподавателя (teacher_bp): предназначен для выполнения текущих рабочих задач, связанных с ведением учебной документации. Данный блок предоставляет интерфейс для регистрации посещаемости студентов в режиме реального времени, а также обеспечивает навигацию по дисциплинам и группам, закрепленным за конкретным сотрудником в рамках текущего расписания.

Преимущества модульной архитектуры:

1. Принцип разделения ответственности (Separation of Concerns): Каждая функциональная область системы изолирована в рамках собственного набора маршрутов и представлений. Это упрощает диагностику: при возникновении сбоев в работе журнала посещаемости область поиска неисправностей ограничивается модулем teacher_bp, что существенно сокращает время на отладку.
2. Масштабируемость: Архитектурное решение допускает бесшовную интеграцию новых компонентов. В частности, предусмотрена возможность внедрения модуля student_bp (личный кабинет обучающегося), который будет функционировать как автономная единица, обращаясь к слою данных через общую реализацию ORM.
3. Разграничение прав доступа: Механизм Blueprint позволяет применять политики безопасности на уровне отдельных модулей. Использование централизованных декораторов прав доступа для всего блока исключает риск пропуска отдельных URL-адресов при настройке системы безопасности.
4. Оптимизация командной разработки: Модульная структура кода минимизирует вероятность возникновения конфликтов слияния (merge conflicts) при совместной работе над проектом, позволяя разработчикам вести работу над отдельными компонентами параллельно.

Использование данной архитектуры обеспечивает гибкость системы: структура приложения является масштабируемой и адаптируемой к изменению бизнес-требований или расширению функциональных возможностей в процессе эксплуатации.

2.2.2. Инициализация и жизненный цикл приложенияФайл app.py является центральным модулем системы, отвечающим за инициализацию параметров конфигурации, формирование контекста выполнения и запуск сервера. Алгоритм запуска приложения реализован как последовательность взаимозависимых этапов, что обеспечивает готовность всех компонентов системы к обработке запросов непосредственно после завершения процесса инициализации. Жизненный цикл запуска системы включает следующие этапы:

1. Конфигурация параметров системы: Инициализация начинается с загрузки критически важных переменных окружения. В их число входит SECRET_KEY, используемый для криптографического подписания сессионных cookie-файлов, что обеспечивает защиту от несанкционированной подделки сессий, а также SQLALCHEMY_DATABASE_URI, задающий параметры подключения к файлу базы данных center.db. Вынесение конфигурационных данных в отдельные переменные обеспечивает необходимую гибкость: адаптация системы под различные среды развертывания (тестовую, производственную) может быть выполнена путем изменения настроек окружения без необходимости модификации исходного кода бизнес-логики.
2. Инициализация расширений: Объект app служит базовым контейнером для интеграции функциональных расширений Flask. Процесс инициализации включает следующие этапы:
   - Интеграция SQLAlchemy (ORM): инициализация объекта БД выполняется через вызов метода db.init_app(app), что обеспечивает привязку экземпляра базы данных к текущему контексту приложения и позволяет управлять соединениями в рамках жизненного цикла системы.
   - Настройка LoginManager: осуществляется конфигурирование менеджера сессий, включая регистрацию функции user_loader.

   Данный механизм обеспечивает автоматическое восстановление состояния объекта пользователя из базы данных при обработке каждого HTTP-запроса, основываясь на уникальном идентификаторе, хранящемся в сессионных данных.
3. Регистрация функциональных модулей (Blueprints): Завершающим этапом инициализации является регистрация модулей посредством метода app.register_blueprint(). Данная процедура осуществляет интеграцию независимых компонентов (auth_bp, admin_bp, teacher_bp) в рамках единого контекста приложения. В результате регистрации система получает информацию о маршрутизации всех функциональных блоков, что позволяет приложению корректно обрабатывать входящие HTTP-запросы в соответствии с заданной структурой URL-адресов.
4. Создание и миграция схемы базы данных: Ключевым этапом жизненного цикла приложения является выполнение блока with app.app_context(): db.create_all(). Данная программная конструкция обеспечивает автоматическую проверку структуры базы данных при инициализации системы. В случае отсутствия необходимых таблиц в файле center.db система инициирует процесс их автоматического создания. Реализация данного механизма позволяет отказаться от ручного выполнения SQL-скриптов при развертывании архитектуры базы данных, что обеспечивает готовность системы к эксплуатации непосредственно после завершения процесса первичной настройки.

Данный подход к инициализации приложения не только обеспечивает высокую надежность запуска, но и создает строгую предсказуемость поведения системы. Каждый этап процесса инициализации логически отделен, что позволяет проводить модульное тестирование конфигурации и упрощает процесс отладки при изменении параметров окружения.

2.2.3. Механизмы безопасности и ролевой модели (RBAC)Безопасность системы базируется на многоуровневом подходе, интегрирующем проверку сессий и ролевое управление (Role-Based Access Control):

- Первый уровень (Аутентификация): Использование библиотеки Flask-Login и декоратора @login_required обеспечивает защиту маршрутов от неавторизованного доступа.
- Второй уровень (Авторизация по ролям): Внутри функций-обработчиков реализована логика проверки атрибута current_user.role.

.Данный механизм, реализованный, в частности, в admin.py, исключает возможность выполнения административных функций (например, создания учетных записей через employees_list) при попытке доступа со стороны пользователей с ролями teacher или student, обеспечивая соблюдение политик информационной безопасности образовательного центра.

2.2.4. Жизненный цикл обработки HTTP-запросаВзаимодействие между модулями информационной системы осуществляется через единый контекст приложения. Процесс обработки HTTP-запроса организован как строго упорядоченный конвейер, обеспечивающий соблюдение безопасности и корректность выполнения бизнес-логики:

1. Маршрутизация (Routing): при обращении пользователя к системе, Flask анализирует входящий URL-адрес и сопоставляет его с зарегистрированными маршрутами. На этом этапе определяется соответствующий «чертеж» (Blueprint), который содержит необходимую функцию-обработчик. Это позволяет эффективно декомпозировать систему на независимые модули, сохраняя чистоту адресного пространства.
2. Анализ контекста и сессии: Система идентифицирует субъекта запроса посредством механизма Flask-Login. Идентификатор пользователя извлекается из защищенных cookie-файлов, после чего происходит десериализация объекта current_user из базы данных. Это позволяет верифицировать права доступа до инициализации основной бизнес-логики.
3. Контроль доступа (Security Gate): Реализована проверка полномочий посредством декораторов (например, @login_required). При отсутствии у пользователя соответствующих прав выполнение запроса блокируется, что предотвращает несанкционированный доступ к защищенным ресурсам.
4. Выполнение бизнес-логики: При успешной верификации вызывается целевая функция модуля. Взаимодействие с данными осуществляется исключительно через ORM-слой, что гарантирует целостность center.db.
5. Формирование ответа: Результат выполнения логики трансформируется в ответ Response. В зависимости от задачи система отдает HTML-документ (рендеринг через Jinja2) или выполняет перенаправление. По завершении этапа контекст запроса очищается, предотвращая утечки данных между сессиями.

Данная архитектурная модель обеспечивает строгую изоляцию полномочий: пользователь взаимодействует исключительно с функционалом, необходимым для выполнения его должностных задач. Поэтапная инкапсуляция процессов обработки снижает риск возникновения критических уязвимостей (включая эскалацию привилегий) и гарантирует стабильность системы при параллельной обработке запросов.

2.3. Проектирование и реализация пользовательского интерфейса2.3.1. Принципы построения интерфейса и концепция UXВизуальная оболочка информационной системы является связующим звеном между сложной серверной логикой и конечным пользователем. Проектирование интерфейса базировалось на концепции User-Centered Design (дизайн, ориентированный на пользователя), где главной задачей является обеспечение максимальной эффективности выполнения профессиональных задач при минимальных когнитивных затратах.

При разработке были соблюдены три фундаментальных принципа:

1. Эргономичность: Расположение элементов управления (форм, кнопок, таблиц) соответствует логике повседневных операций пользователя. Часто используемые функции вынесены на первый план.
2. Интуитивность: Использование общепринятых паттернов взаимодействия позволяет сократить время адаптации пользователей к системе, исключая необходимость в обучении.
3. Лаконичность: Архитектура интерфейса ориентирована на отсутствие визуальной избыточности, что акцентирует внимание пользователя на целевых данных (учебном расписании, сведениях о группах и показателях успеваемости).

Концепция информационной панели (Dashboard)

Для обеспечения высокой скорости обработки данных в условиях интенсивной операционной нагрузки был выбран паттерн «Dashboard». Данное архитектурное решение обладает следующими преимуществами:

- Контекстная доступность: Минимизация количества переходов между страницами позволяет пользователю видеть общую картину (например, состояние групп или текущую нагрузку преподавателя) на одном экране.
- Приоритезация функций: Организация «быстрого доступа» к критически важным операциям сокращает операционные издержки, способствуя росту эффективности работы преподавательского состава.

Визуальная иерархия и эргономика

При проектировании экранных форм применялись следующие методы визуализации:

- Модульность: Использование карточной структуры (Cards) обеспечивает четкое разграничение функциональных доменов и облегчает восприятие структуры данных.
- Цветовое кодирование: Применение цветовой дифференциации статусов (например, индикация посещаемости) позволяет мгновенно интерпретировать состояние записи без обращения к текстовым метаданным.
- Согласованность: Единообразие графического оформления и паттернов поведения элементов управления обеспечивает предсказуемость отклика системы во всех программных модулях.

Реализованный подход позволяет трансформировать программный продукт из самостоятельного инструмента в прозрачный вспомогательный механизм, оптимизирующий выполнение рутинных задач по обработке данных и позволяющий пользователям сфокусироваться на образовательном процессе.

2.3.2. Механизм динамической шаблонизации на базе Jinja2Основным архитектурным решением при проектировании уровня представления системы является использование движка шаблонов Jinja2. В отличие от статических HTML-документов, данный инструмент обеспечивает динамический рендеринг контента, интегрируя бизнес-логику в процесс формирования страниц. Система осуществляет динамическую генерацию контента на основе актуальных данных из БД и контекста авторизованного пользователя. Такой подход позволяет реализовывать персонализированный интерфейс, в котором наполнение страницы адаптируется под полномочия субъекта доступа и текущее состояние информационной базы.

Преимущества и архитектурные решения, реализованные в проекте:

1. Механизм наследования шаблонов (Template Inheritance)

Для проектирования интерфейса выбран паттерн наследования шаблонов, реализованный посредством формирования мастер-шаблона base.html. Данный файл определяет структуру каркаса системы, включая глобальные мета-данные, подключение внешних библиотек стилей и сценариев, а также статические элементы навигации и футера. Специфические представления страниц (например, dashboard_admin.html, attendance.html) используют директиву {% extends "base.html" %}, переопределяя лишь целевые области контента через блоки {% block content %}.

Данный архитектурный подход обеспечивает строгое соблюдение принципа DRY (Don't Repeat Yourself), исключая дублирование исходного кода верстки. Это не только упрощает сопровождение системы, но и гарантирует визуальное единообразие всех разделов приложения, минимизируя вероятность рассогласования интерфейса при внесении глобальных правок.

2. Автоматизация формирования контента

Использование итераторов и встроенных управляющих конструкций Jinja2 (например, {% for item in items %}) позволило автоматизировать процесс генерации табличных представлений данных. В данной системе этот механизм обеспечивает динамическую отрисовку списков обучающихся, учебных групп и графиков занятий.

Реализованный подход гарантирует, что любое изменение в базе данных SQLite транслируется в интерфейс приложения в режиме реального времени. Это исключает необходимость внесения правок в структуру HTML-разметки при обновлении контента, что существенно снижает риск рассинхронизации данных и представления, а также минимизирует вероятность ошибок при манипуляции с интерфейсом.

3. Логическое ветвление и адаптивное отображение интерфейса

Механизм шаблонизации Jinja2 обеспечивает поддержку условных конструкций, позволяющих динамически модифицировать структуру DOM-дерева в зависимости от контекста выполнения запроса. Использование выражений вида {% if current_user.role == 'admin' %} позволяет реализовать адаптивную логику отображения, обеспечивая контекстную доступность элементов управления (панелей администрирования, кнопок редактирования и меню).

Реализованный метод выполняет две критические функции:

- Безопасность (Security by obscurity/UI-level protection): предотвращает возможность непреднамеренного взаимодействия пользователей с недоступными для их роли функциональными узлами.
- Эргономика (UX-оптимизация): очищает интерфейс от избыточных элементов, что снижает когнитивную нагрузку на пользователя и делает навигацию более предсказуемой.

Таким образом, адаптивное представление является логическим дополнением к серверной авторизации, обеспечивая целостность интерфейса при работе различных категорий пользователей в едином информационном пространстве.

4. Фильтрация и нормализация данных (Filters)

Одним из функциональных инструментов Jinja2 является механизм фильтрации, позволяющий выполнять предобработку данных непосредственно перед их передачей на уровень представления. В рамках данной системы фильтры применяются для нормализации форматов вывода (например, дат, числовых значений или строковых данных), обеспечивая их приведение к стандартам локализации.

Данный подход обеспечивает следующие преимущества:

- Интерфейсная читаемость: единообразное представление данных вне зависимости от их формата хранения в базе данных.
- Разделение ответственности: бизнес-логика остается сфокусированной на хранении и обработке «чистых» данных, в то время как уровень представления берет на себя задачу по их адаптации для конечного пользователя.

Использование фильтров минимизирует дублирование кода форматирования в различных частях приложения, повышая единообразие и надежность пользовательского интерфейса.

Значение для архитектурной целостности системы:

Использование Jinja2 трансформирует процесс разработки пользовательского интерфейса из статичной верстки в декларативное программирование шаблонов. Основным преимуществом данной концепции является высокая степень слаженности компонентов: изменения в схеме данных SQLAlchemy автоматически транслируются в уровень представления (Jinja2), что исключает рассинхронизацию между структурой модели и интерфейсом.

Данный подход снижает совокупную стоимость владения системой за счет минимизации трудозатрат на поддержку и сопровождение фронтенд-компонентов. Обеспечивая строгую интеграцию серверной бизнес-логики и уровня представления, описываемый метод гарантирует высокую надежность интерфейса и гибкость системы при внесении последующих архитектурных модификаций.

2.3.3. Обеспечение интерактивности средствами JavaScriptФункционирование клиентской части информационной системы дополнено сценариями на языке JavaScript, что позволяет повысить отзывчивость интерфейса и оптимизировать взаимодействие пользователя с системой. Использование клиентских скриптов продиктовано необходимостью обработки событий в браузере без инициации полного цикла перезагрузки страниц, что сокращает время ожидания отклика и уменьшает когнитивную нагрузку при работе с навигацией и формами.

Применение данных программных средств позволило реализовать следующие функциональные возможности:

- Динамическая модификация DOM: изменение отображаемых данных и элементов управления в ответ на действия пользователя в режиме реального времени.
- Локальная обработка данных: проведение первичной валидации вводимых сведений непосредственно в браузере, что снижает нагрузку на серверную часть и минимизирует количество некорректных HTTP-запросов.
- Асинхронное взаимодействие: реализация механизмов обмена данными с сервером (AJAX/Fetch API), обеспечивающих плавный переход между состояниями приложения без необходимости обновления всей страницы.

Таким образом, клиентский уровень JavaScript выступает в качестве инструментария для повышения эргономики системы, дополняя серверные возможности шаблонизатора Jinja2 и обеспечивая требуемый уровень оперативного взаимодействия с интерфейсом.

2.3.4. Функциональная структура экранных формПроектирование экранных форм информационной системы осуществлялось с учетом специфики профессиональной деятельности пользователей. Архитектура интерфейса базируется на принципах эргономики, предусматривающих минимизацию избыточного ручного ввода и строгую логическую группировку управляющих элементов. Данный подход обеспечивает соответствие между операциями пользователя и транзакциями в базе данных, предотвращая накопление некорректной информации.

В таблице 2.3 представлена классификация основных интерфейсных модулей, формирующих клиентский уровень системы:

Реализованная структура экранных форм позволяет разделить пользовательские сценарии на функциональные домены, обеспечивая целостность данных на всех этапах жизненного цикла обработки запросов. Использование типизированных модулей упрощает масштабирование системы: при добавлении новых сущностей разработчик использует уже апробированные шаблоны проектирования, что поддерживает единообразие взаимодействия на всех уровнях приложения.

2.3.5. Анализ проектных решений в проектировании экранных формВ ходе проектирования экранных форм были реализованы следующие архитектурные принципы, направленные на повышение надежности и эффективности взаимодействия с информационной системой:

- Функциональная специализация: Структура каждой экранной формы оптимизирована для выполнения конкретного бизнес-процесса. Например, модуль «Расписание» реализован с учетом принципа функциональной изоляции: минимизация второстепенных элементов управления позволяет преподавателю сфокусироваться на целевой задаче (отметка посещаемости), что снижает вероятность непреднамеренного воздействия на конфигурационные параметры системы.
- Динамическая связность компонентов. Элементы управления (селекторы групп, филиалов) реализованы как динамические компоненты, работающие в связке с реляционной структурой базы данных SQLite. Это обеспечивает актуальность данных в режиме реального времени: любые изменения в административном контуре (добавление сущностей) автоматически ретранслируются во все зависимые интерфейсные модули, исключая рассогласование состояний системы.
- Обеспечение целостности данных (Data Integrity): При проектировании форм ввода применен подход типизации пользовательского ввода. Использование выпадающих списков (Dropdowns) вместо текстовых полей для критически важных атрибутов (например, привязка преподавателя к дисциплине) позволяет ограничить область допустимых значений (Domain Constraint). Это является превентивной мерой против логических ошибок и аномалий в базе данных, гарантируя соблюдение ссылочной целостности на уровне интерфейса.

Реализованная архитектура форм обеспечивает прозрачность бизнес-процессов образовательного центра. Реализованное разделение интерфейса на функциональные домены позволяет дифференцировать доступ к информации в соответствии с ролевой моделью: обучающиеся получают доступ к актуальным графикам обучения, преподаватели — к инструментам оперативного контроля, а администраторы — к полнофункциональным средствам управления ресурсами системы.

2.3.6. Организация обработки форм и верификация входных данныхБезопасность и целостность информационной системы напрямую зависят от качества механизмов обработки данных, поступающих от пользователя. Архитектурно процесс передачи данных от клиентской части к серверу реализован посредством протокола HTTP с применением метода POST. Использование данного метода обеспечивает инкапсуляцию передаваемых параметров в теле запроса, что повышает безопасность взаимодействия за счет исключения возможности их отображения в адресной строке браузера.

Технологические уровни верификации

Проектирование подсистемы обработки данных базируется на многоуровневой схеме контроля:

1. Клиентская верификация: На этапе формирования запроса применяются стандартные атрибуты HTML5 (например, required, type), позволяющие провести первичную фильтрацию ввода. Данный подход снижает долю избыточного трафика, исключая отправку некорректных запросов до момента их передачи на серверную сторону.
2. Серверная валидация: Является основным эшелоном обеспечения консистентности данных. Каждая функция-обработчик (view-function) во фреймворке Flask реализует проверку входящей информации. Например, при регистрации учетной записи осуществляется запрос User.query.filter_by(...) для верификации уникальности идентификатора, что предотвращает возникновение логических коллизий в базе данных.
3. Безопасность сессий и защита от атак: Для минимизации рисков несанкционированной подмены запросов (CSRF) применяются штатные механизмы управления сессиями Flask с использованием защищенного ключа SECRET_KEY. В критических узлах ввода проводится санитарная очистка данных от специальных символов, что выступает превентивной мерой против инъекционных атак.

Механизм оповещения пользователя (Feedback Loop)

В качестве инструмента обратной связи реализована система Flash-сообщений. При возникновении исключительных ситуаций (неверные учетные данные, нарушение ссылочной целостности при удалении записи, ошибки форматирования дат) система прерывает выполнение операции и инициирует формирование контекстно-зависимого уведомления.

Реализованный подход трансформирует экранные формы в интеллектуальные узлы управления данными. Система функционирует как отказоустойчивая среда: невозможность сохранения информации, нарушающей бизнес-логику или целостность связей, гарантирует стабильность хранения данных. Интеграция механизмов валидации с системой визуальных уведомлений повышает удобство эксплуатации (UX) для персонала, обеспечивая четкое понимание причин отказа при совершении некорректных действий.

2.4. Тестирование и оценка эффективности реализации системыПроцесс тестирования является неотъемлемым этапом жизненного цикла разработки ПО (SDLC), обеспечивающим гарантию того, что созданная система соответствует функциональным требованиям и демонстрирует устойчивость к нештатным ситуациям.

2.4.1. Методология обеспечения качества и уровни тестирования программного обеспеченияДля достижения требуемых показателей надежности информационной системы была реализована стратегия тестирования, базирующаяся на концепции «пирамиды тестирования» (Testing Pyramid). Применение данного подхода позволило обеспечить многоуровневый контроль качества на всех этапах жизненного цикла разработки (SDLC) и минимизировать риски регрессионных ошибок при масштабировании функционала.

Процесс обеспечения качества был разделен на три иерархичных уровня:

1. Модульное тестирование (Unit Testing): На данном уровне проводилась верификация минимальных функциональных единиц программного кода. Основным объектом тестирования выступала бизнес-логика: алгоритмы ролевой авторизации, методы нормализации форматов данных и функции хеширования учетных данных. Использование стандартного модуля unittest позволило автоматизировать контроль возвращаемых значений функций, гарантируя, что локальные модификации кода не нарушают работу зависимых компонентов.
2. Интеграционное тестирование: Этап был сфокусирован на анализе взаимодействия между независимыми архитектурными узлами: компонентами приложения (Blueprints) и уровнем абстракции базы данных (SQLAlchemy). Ключевой задачей являлось подтверждение корректности передачи данных между серверными контроллерами и слоем представления. Особое внимание уделялось проверке реляционной целостности: верификации каскадных операций (например, удаление записей в таблице Attendance при аннулировании сущности пользователя), что гарантирует непротиворечивость данных при выполнении CRUD-операций.
3. Приемочное тестирование (UAT — User Acceptance Testing): Финальный этап контроля качества, проводимый в условиях, идентичных эксплуатационным. Целью тестирования являлась валидация соответствия реализованного функционала требованиям конечных пользователей (администраторов, преподавателей и обучающихся). На данном этапе отрабатывались сквозные (End-to-End) сценарии: от авторизации до регистрации посещаемости, включая проверку ролевой разграниченности доступа к интерфейсным узлам.

Реализованная методология заложила основу для построения «тестируемого кода» (testable code). Строгое разделение ответственности между модулями и модульная архитектура позволяют проводить регрессионное тестирование при внесении изменений в систему. Такой подход обеспечивает высокую степень предсказуемости поведения приложения при различных паттернах пользовательского взаимодействия, существенно снижая вероятность критических отказов в информационной среде образовательного центра.

2.4.2. Матрица сценариев функционального тестированияДля комплексной оценки корректности функционирования системы была разработана матрица тестовых сценариев (Test Case Matrix). Данный перечень включает как позитивные сценарии, верифицирующие целевой функционал, так и негативные, направленные на проверку отказоустойчивости при вводе некорректных данных и попытках несанкционированного доступа к защищенным ресурсам.

Результаты проведенных испытаний подтверждают функциональную стабильность системы на всех ключевых участках взаимодействия. На основании анализа матрицы тестирования можно сформулировать следующие выводы:

1. Устойчивость к аномальным входным данным: Тесты TC-04 и TC-12 доказывают эффективность механизмов валидации, предотвращающих запись некорректных сведений в базу данных и блокирующих попытки выполнения нештатных операций.
2. Эффективность реализации модели RBAC (Role-Based Access Control): Проверка сценариев TC-03, TC-07 и TC-09 подтверждает строгое разграничение полномочий. Изоляция функциональных доступов полностью исключает риски нарушения конфиденциальности и целостности данных между ролевыми группами.
3. Поддержание ссылочной целостности: Успешное выполнение сценария TC-10 подтверждает корректность настройки каскадных правил в схеме базы данных. Использование механизма ON DELETE CASCADE исключает появление логически изолированных записей, обеспечивая согласованность БД даже при проведении сложных административных манипуляций.

Таким образом, реализованный комплекс сценариев охватывает все функциональные модули приложения, подтверждая его готовность к эксплуатации и высокий уровень надежности в условиях реальной информационной среды образовательного центра.

2.4.3. Оценка эффективности и качественные показатели системыЭффективность внедренной информационной системы определялась через сопоставление целевых показателей автоматизации с качественными изменениями в бизнес-процессах образовательного центра. Переход от разрозненных методов учета к централизованной модели данных позволил систематизировать информационные потоки. Анализ результативности реализованного решения проводится на основе следующих критериев:

- Оперативность обработки данных: Централизация БД исключила необходимость дублирования информации в различных подразделениях, что сократило среднее время выполнения административных транзакций (внесение записей, формирование отчетности).
- Согласованность и достоверность сведений: Использование единой реляционной модели данных в сочетании с жесткими алгоритмами валидации позволило исключить несогласование информации между модулями системы. Интеграция автоматических проверок на этапе ввода предотвращает появление логических несоответствий и дублирующихся данных, гарантируя, что все записи в базе данных соответствуют установленным правилам бизнес-логики.
- Эргономика и доступность: Внедрение ролевой модели доступа (RBAC) обеспечило персонализированное представление данных. Это существенно оптимизировало выполнение узкоспециализированных задач (учет посещаемости преподавателем, планирование графика обучающимся), снизив время на поиск необходимого функционала в интерфейсе.
- Масштабируемость сопровождения: Переход к декларативному описанию представлений через Jinja2 и модульной структуре кода упростил внесение последующих правок. Время, затрачиваемое на поддержку фронтенд-компонентов, сократилось за счет использования паттернов наследования шаблонов (DRY).

Результаты качественного анализа свидетельствуют о том, что автоматизация не только упростила операционную деятельность сотрудников, но и сформировала технологический базис для дальнейшего развития информационной инфраструктуры центра.

2.5 Выводы по второй главе

На основании проведенного комплекса испытаний, анализа архитектурных решений и оценки качественных показателей можно констатировать, что разработанная информационная система соответствует предъявляемым техническим требованиям и готова к внедрению в опытную эксплуатацию.

Реализованная архитектурная модель, объединяющая возможности объектно-реляционного отображения и механизмы гранулярного ролевого доступа, позволила достичь ключевых целей автоматизации:

- Оптимизация операционной деятельности: минимизация трудозатрат на рутинные процедуры обработки учебной информации.
- Обеспечение структурной целостности: внедрение превентивных механизмов валидации гарантирует достоверность данных на всех этапах жизненного цикла записей.
- Прозрачность взаимодействия: создание унифицированной среды, обеспечивающей разграниченный доступ к функциональным модулям для всех участников образовательного процесса.

Архитектурный базис системы обладает достаточным уровнем абстракции для дальнейшего масштабирования. Модульная структура приложения позволяет интегрировать дополнительные функциональные блоки, не требуя рефакторинга ядра системы, что обеспечивает долгосрочную устойчивость и адаптивность решения к изменяющимся требованиям образовательного центра.

ЗАКЛЮЧЕНИЕВ ходе выполнения выпускной квалификационной работы была спроектирована и реализована информационная система для автоматизации бизнес-процессов управления образовательным центром. Разработанный программный комплекс обеспечивает комплексную поддержку деятельности административного состава, преподавателей и обучающихся.

В рамках работы были решены следующие задачи:

1. Анализ предметной области: Проведен детальный анализ бизнес-процессов образовательного центра, выявлены ключевые проблемы управления и обоснованы функциональные требования, необходимые для повышения эффективности работы организации.
2. Проектирование архитектуры и модели данных: Спроектирована масштабируемая модульная архитектура на основе веб-фреймворка Flask, а также разработана реляционная модель базы данных (SQLAlchemy, SQLite), реализующая принцип «Единого источника истины» (SSOT) для обеспечения целостности и достоверности учебной информации.
3. Программная реализация: Осуществлена разработка функциональных модулей системы, внедрены механизмы аутентификации пользователей и настроена ролевая модель доступа (RBAC), обеспечивающая строгое разграничение прав для администраторов, преподавателей и обучающихся.
4. Тестирование программного продукта: Проведено комплексное функциональное тестирование, подтвердившее корректность работы всех модулей, устойчивость к несанкционированному доступу, а также соответствие заданным требованиям к надежности и производительности.

Научная и практическая значимость работы заключается в создании автоматизированного инструмента, который переводит документооборот образовательного центра в цифровую среду. Реализованная система минимизирует влияние «человеческого фактора» на достоверность отчетности и обеспечивает прозрачное взаимодействие всех участников образовательного процесса.

Перспективы дальнейшего развития системы включают внедрение модулей аналитической отчетности, интеграцию системы уведомлений для обучающихся, а также возможность миграции на высоконагруженные СУБД (например, PostgreSQL) при расширении филиальной сети.

Разработанная информационная система полностью готова к опытной эксплуатации и может быть внедрена в практику работы образовательного центра в качестве основного инструмента автоматизации управления учебной деятельностью.

8,9 баллов из 10 по шкале Главреда

369 предложений
5905 слов, 55056 знаков

403 стоп-слова. Основные проблемы: Страдательный залог причастие неуклюжий синтаксис слишком коротко нет глагола

Максим Ильяхов, Анатолий Буров. С 1 декабря 2014 года. Пишите: mail@glvrd.ru

ИП Ильяхов Максим Олегович. ИНН: 231214502283; ОГРНИП: 317237500352990. Почта: maxim.ilyahov@yandex.ru